

(<https://databricks.com>)

Decision Trees for handwritten digit recognition

This notebook demonstrates learning a Decision Tree (https://en.wikipedia.org/wiki/Decision_tree_learning) using Spark's distributed implementation. It gives the reader a better understanding of some critical hyperparameters (https://en.wikipedia.org/wiki/Hyperparameter_optimization) for the tree learning algorithm, using examples to demonstrate how tuning the hyperparameters can improve accuracy.

This notebook uses the classic MNIST handwritten digit recognition dataset.

Load MNIST training and test datasets

The datasets consist of vectors of pixels representing images of handwritten digits. These datasets are stored in the popular LibSVM dataset format. This notebook uses MLlib's LibSVM dataset reader utility to load the datasets.

```
val training = spark.read.format("libsvm").load("/databricks-datasets/mnist-digits/data-001/mnist-digits-train.txt")
val test = spark.read.format("libsvm").load("/databricks-datasets/mnist-digits/data-001/mnist-digits-test.txt")

// Cache data for multiple uses.
training.cache()
test.cache()

println(s"There are ${training.count} training images and ${test.count} test images.")
```

Display the data. Each image has the true label (the `label` column) and a vector of `features` which represent pixel intensities.

```
display(training)
```

Train a Decision Tree

Begin by training a decision tree using the default settings. Before training, you use the `StringIndexer` class to tell the algorithm that the labels are categories 0-9, instead of continuous values. Then, you use a `Pipeline` to connect the feature preprocessing with the decision tree algorithm. ML Pipelines are Spark tools for linking Machine Learning algorithms into workflows. To learn more about Pipelines, see the other ML example notebooks in Databricks and the ML Pipelines user guide (<http://spark.apache.org/docs/latest/ml-guide.html>).

```
// Import the ML algorithms.
import org.apache.spark.ml.classification.{DecisionTreeClassifier, DecisionTreeClassificationModel}
import org.apache.spark.ml.feature.StringIndexer
import org.apache.spark.ml.Pipeline

// StringIndexer: Read input column "label" (digits) and annotate them as categorical values.
val indexer = new StringIndexer().setInputCol("label").setOutputCol("indexedLabel")
// DecisionTreeClassifier: Learn to predict column "indexedLabel" using the "features" column.
val dtc = new DecisionTreeClassifier().setLabelCol("indexedLabel")
// Chain indexer + dtc together into a single ML Pipeline.
val pipeline = new Pipeline().setStages(Array(indexer, dtc))
```

Now fit a model to the data.

```
val model = pipeline.fit(training)
```

Use Databricks ML visualization to inspect the learned tree. Visualization is only available for some models.

```
// The tree is the last stage of the Pipeline. Display it!
val tree = model.stages.last.asInstanceOf[DecisionTreeClassificationModel]
display(tree)
```

You can see above how the tree makes predictions. When classifying a new example, the tree starts at the "root" node (at the top). Each tree node tests a pixel value and goes either left or right. At the bottom "leaf" nodes, the tree predicts a digit as the image's label.

Exploring "maxDepth": training trees of different sizes

This section tunes a single hyperparameter `maxDepth`, which determines how deep (and large) the tree can be. It trains trees at varying depths and see how it affects the accuracy on the held-out test set.

Note: The next cell can take about 1 minute to run since it is training several trees which get deeper and deeper.

```
val variedMaxDepthModels = (0 until 8).map { maxDepth =>
  // For this setting of maxDepth, learn a decision tree.
  dtc.setMaxDepth(maxDepth)
  // Create a Pipeline with the feature processing stage (indexer) plus the tree algorithm
  val pipeline = new Pipeline().setStages(Array(indexer, dtc))
  // Run the ML Pipeline to learn a tree.
  pipeline.fit(training)
}

// Define an evaluation metric. This example uses "weightedPrecision", which is equivalent to 0-1 accuracy.
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
val evaluator = new MulticlassClassificationEvaluator().setLabelCol("indexedLabel").setMetricName("weightedPrecision")

// For each maxDepth setting, make predictions on the test data, and compute the accuracy metric.
val accuracies = (0 until 8).map { maxDepth =>
  val model = variedMaxDepthModels(maxDepth)
  // Calling transform() on the test set runs the fitted pipeline.
  // The learned model makes predictions on each test example.
  val predictions = model.transform(test)
  // Calling evaluate() on the predictions DataFrame computes the accuracy metric.
  (maxDepth, evaluator.evaluate(predictions))
}.toDF("maxDepth", "accuracy")
```

You can display accuracy results and see immediately that deeper, larger trees are more powerful classifiers, achieving higher accuracies.

Note: When you run `display()`, you will get a table. Click on the plot icon below the table to create a plot, and use "Plot Options" to adjust what is displayed.

```
display(accuracies)
```

Even though deeper trees are more powerful, they are not always better. As tree depth increases, training takes longer. You also risk overfitting (<https://en.wikipedia.org/wiki/Overfitting>) (fitting the training data so well that performance decreases on test data); it is important to tune parameters *based on held-out data* (https://en.wikipedia.org/wiki/Test_set) to prevent overfitting.

Exploring `maxBins` : discretization for efficient distributed computing

This section explores a more expert-level setting `maxBins`. For efficient distributed training of Decision Trees, Spark and most other libraries discretize (or "bin") continuous features (such as pixel values) into a finite number of values. This is an important step for the distributed implementation, but it introduces a tradeoff: Larger `maxBins` mean your data will be more accurately represented, but it will also mean more communication (and slower training).

The default value of `maxBins` generally works, but it is interesting to explore on the handwritten digit dataset. The images are in grayscale. Setting `maxBins = 2`, effectively makes it a black-and-white image. Does that affect the accuracy of the model?

```
dtc.setMaxDepth(6) // Set maxDepth to a reasonable value.
val accuracies = Seq(2, 4, 8, 16, 32).map { case maxBins =>
  // For this value of maxBins, learn a tree.
  dtc.setMaxBins(maxBins)
  val pipeline = new Pipeline().setStages(Array(indexer, dtc))
  val model = pipeline.fit(training)
  // Make predictions on test data, and compute accuracy.
  val predictions = model.transform(test)
  (maxBins, evaluator.evaluate(predictions))
}.toDF("maxBins", "accuracy")
display(accuracies)
```

The extreme discretization (black and white) decreases accuracy, but only a bit. Using more bins increases the accuracy, but also makes learning more costly.

What's next?

- **Explore:** Try out tuning other parameters of trees---or even ensembles like Random Forests or Gradient-Boosted Trees (<http://spark.apache.org/docs/latest/ml-classification-regression.html#tree-ensembles>).
- **Automated tuning:** This type of tuning does not have to be done manually. (It is done manually here to show the effects of tuning in detail.) MLlib provides automated tuning functionality using `CrossValidator`. See the other Databricks ML Pipeline guides or the Spark ML user guide (<http://spark.apache.org/docs/latest/ml-guide.html>) for details.